

[바로가기]

- [Frontend 실행 방법 \(Bun\)](#)
 - [Weather Backend 실행 방법 \(Python\)](#)
 - [기존 작업 절차](#)
 - **중요!**
-

Frontend 실행 방법 (Bun)

1. Bun 설치 (설치되어 있지 않은 경우)

- Windows (PowerShell):

```
powershell -c "irm bun.sh/install.ps1 | iex"
```

2. 의존성 설치

- `frontend` 디렉토리로 이동 후 설치:

```
cd frontend  
bun install
```

3. 개발 서버 실행

```
bun dev
```

Weather Backend 실행 방법 (Python)

1. 가상환경 생성 및 활성화

- `weather` 디렉토리로 이동:

```
cd weather
```

- 가상환경 생성:

```
python -m venv venv
```

- 활성화 (Windows):

```
.\venv\Scripts\activate
```

2. 패키지 설치

```
pip install -r requirements.txt
```

3. 서버 실행

```
python main.py
```

또는 FastAPI API 직접 실행:

```
uvicorn api.app:app --reload
```

기존 작업 절차

절차 등 **branch**를 나눠서 작업할 예정입니다.

중요!

- 기능 추가는 **Pull requests** 기능을 이용하세요! 한 파일을 다수가 동시에 수정하면 안됩니다!
- **main** 브랜치는 현재 상태로 유지합니다. 배포할 소스코드만 업로드합니다.
- 모든 작업은 이 브랜치(**dev**)에서 끝납니다. Merge, push, commit 등 모든 액션에서 확인 후 시작하세요!
- 이 Repository는 공개되지 않습니다. 종료 전까지 공개하지 않을 예정이오나 사업단 또는 그 외 증빙이 필요할 경우 공개될 수 있습니다.

1. 로컬 동기화: 작업 시작 전 원격 **dev** 브랜치의 최신 상태를 로컬로 가져옴

```
git checkout dev  
git pull origin dev
```

2. 기능 브랜치(Feature Branch) 생성: 각 작업자는 **dev** 브랜치에서 분기하여 독립적인 작업 브랜치 생성.

- 명명 규칙(**feature/login**, **feature/api-setup**) 적용.

```
git checkout -b feature/작업명
```

3. 독립 작업 및 커밋: 할당된 기능 구현 후 로컬 환경에 커밋.

```
git add .  
git commit -m "작업 내용 명시"
```

4. 원격 푸시: 작업이 완료된 기능 브랜치를 원격 저장소에 업로드.

```
git push origin feature/작업명
```

5. Pull Request (PR) 생성: GitHub 웹 인터페이스에서 작업 브랜치(**feature/작업명**)를 타겟 브랜치(**dev**)로 병합 (Merge)해달라고 요청.
6. 코드 리뷰 및 병합: 팀원 검토 후 이상이 없으면 **de**으로 병합. 병합 완료 후 원격 및 로컬의 **feature** 브랜치 삭제.

7. 사이클 반복: 새로운 기능 개발 시 1번 단계부터 다시 수행.

동시 작업 충돌(Conflict) 통제 원칙

- 작업 단위 분리: 개발자 간 동일 파일 수정이 겹치지 않도록 기능 및 컴포넌트 단위를 명확히 분할.
- 주기적 동기화: 작업 기간이 길어질 경우, 원격 `dev`에 업데이트된 다른 팀원의 코드를 자신의 `feature` 브랜치로 수시로 병합하여 릴리스 직전의 대규모 충돌 방지.

```
git pull origin dev
```

- **로컬 충돌 해결:** PR 생성 전 로컬 환경에서 발생하는 충돌을 먼저 해결한 뒤 원격에 반영.